

Interconnection Structures

Condensed Study Notes

Generated: 2026-04-28 22:55 UTC

COMPUTER ARCHITECTURE & ORGANIZATION LECTURE (WEEK 2)

This lecture series delves into Computer Architecture and Organization. In this second week, we begin to explore the fundamental building blocks and operational principles of computers.

At the core of any computer is the CPU (Central Processing Unit). Think of the CPU as the computer's brain – it's responsible for carrying out all the instructions that make a program run. The CPU operates through a continuous cycle, known as the fetch-decode-execute cycle.

Here's a breakdown of that cycle:

- **Fetch:** The CPU retrieves the next instruction from the computer's memory. It's like fetching a recipe card from a cookbook.
- **Decode:** The CPU interprets the instruction to understand what action needs to be performed. This is like reading the recipe and figuring out what ingredients and steps are involved.
- **Execute:** The CPU performs the action specified by the instruction. This is the actual cooking part – mixing ingredients, applying heat, etc.

This cycle repeats billions of times per second. The speed at which this cycle completes is often referred to as the clock speed, typically measured in Gigahertz (GHz). A higher clock speed means the CPU can perform more cycles in a given time. However, it's important to remember that a faster CPU doesn't automatically guarantee a faster overall computer system. Other components, such as memory (where data and instructions are stored) and I/O (Input/Output devices like keyboards and screens), also play crucial roles in system performance.

Interconnection Structures

Computers are built from three essential modules: the Processor (CPU), Memory, and Input/Output (I/O) devices. For these modules to work together and share information, they need pathways to communicate.

The collection of these pathways that connect the processor, memory, and I/O modules is known as the **interconnection structure**. The specific design of this structure is determined by the types of data exchanges that need to occur between these modules. Think of it like a city's road network: the roads (interconnection structure) are designed to allow cars (data) to travel between different locations like homes (memory), offices (processor), and shops (I/O).

This section will explore different ways these modules can be connected, including:

- **Bus structures:** These are pathways that allow multiple components to share a common set of wires. We'll look at single bus and multiple bus configurations.

I/O module

From an internal perspective, the I/O module performs operations that are functionally similar to memory operations. The primary operations for I/O are read and write.

An I/O module can manage multiple external devices. Each interface to an external device is called a port and is assigned a unique address, like 0, 1, or M-1. In addition to these ports, there are dedicated external data pathways for sending and receiving data with external devices.

The interconnection structure needs to support several types of data transfers:

- **Processor to Memory:** The processor writes data to memory.
- **Memory to Processor:** The processor reads instructions or data from memory.
- **Processor to I/O:** The processor sends data to an I/O device.
- **I/O to Processor:** The processor reads data from an I/O device through the I/O module.
- **I/O to or from Memory:** In these cases, the I/O module can directly exchange data with memory without involving the processor. This is known as Direct Memory Access (DMA).

Beyond data transfer, the I/O module also plays a role in system control by sending interrupt signals to the processor. These signals inform the processor about events occurring with I/O devices.

Memory

Memory is where a computer stores data and instructions. Think of it like a vast collection of mailboxes, each with a unique address.

- **Address:** This is the unique numerical label (like 0, 1, up to N-1) assigned to each location in memory. It's how the computer knows exactly where to find or put information.
- **Word:** Memory is organized into "words," which are units of data of a specific length. A word can be read from or written into memory.
- **Control Signals:** To interact with memory, the computer uses control signals like "read" and "write." The "read" signal tells the memory to send data out from a specific address, while the "write" signal tells it to store data at a given address.

A typical memory module contains N words, all of the same length, each identified by its own address.

Complex Instruction Set Architecture (CISC)

Complex Instruction Set Architecture (CISC) is an approach to CPU design that aims to increase performance by making the hardware more complex.

The core idea behind CISC is to accommodate very complex low-level language statements. In this architecture, a single instruction can perform multiple operations that would typically require several separate instructions.

Think of it like a highly specialized tool. Instead of needing separate tools for measuring, cutting, and joining wood, a CISC instruction might be like a sophisticated woodworking machine that can perform all these actions with a single command. For example, a multiplication command in CISC could handle loading the data, performing the multiplication, and then storing the result, all within that one instruction.

Instruction set Architecture: CISC, RISC

The Instruction Set Architecture (ISA) defines the set of commands, or instructions, that a CPU understands and can execute. Think of it as the CPU's vocabulary and grammar.

Previously, we touched on Complex Instruction Set Architecture (CISC). CISC is a design philosophy where individual instructions are very powerful and can perform multiple operations at once. For example, a single CISC instruction might be able to load data from memory, perform a calculation on it, and then store the result back to memory.

This approach aims to make programming easier by allowing complex tasks to be represented by fewer instructions. However, it can also make the CPU design more complex, as it needs to handle these intricate, multi-step instructions.

In contrast, Reduced Instruction Set Architecture (RISC) is a different design philosophy. RISC aims for a smaller, simpler set of instructions, where each instruction performs a very basic, single operation. For instance, a RISC processor might have separate instructions for loading data from memory, performing a calculation, and storing data back to memory. These simple instructions can be executed very quickly and efficiently by the CPU.

This simplicity in the instruction set allows for a more streamlined and faster CPU design, often leading to better performance for common operations. The trade-off is that a complex task might require more individual RISC instructions to accomplish compared to a single CISC instruction. It's like the difference between a Swiss Army knife (CISC) with many tools in one, and a set of specialized tools (RISC) where each tool does one job perfectly.

Reduced Instruction Set Architecture (RISC)

[Content could not be generated for this section.]

The “best” way to design a CPU has been a subject of debate

[Content could not be generated for this section.]

Characteristic of RISC

[Content could not be generated for this section.]

Characteristic of CISC

[Content could not be generated for this section.]

Which of the design method of CISC and RISC is better in designing computer

[Content could not be generated for this section.]

Difference

[Content could not be generated for this section.]

Instruction and Instruction Types

[Content could not be generated for this section.]

Word length

[Content could not be generated for this section.]

INSTRUCTION SEQUENCING

[Content could not be generated for this section.]